MRMS — Multi-Agent Research & Marketing System

> **B.Tech Final-Year Project** | Department of Computer Science
> *Multi-Agent System for Analyzing Digital Marketing Effects on Consumer Buying Behavior*

MRMS is an end-to-end automated research workbench that investigates how **digital marketing exposure** influences **consumer buying behavior**. It replaces manual data gathering and spreadsheet analysis with a resilient, real-time, multi-agent AI pipeline — all visible through an iOS-inspired dark dashboard.

📋 Table of Contents

- System Overview
- Architecture Diagram
- Frontend
- Backend
- AI & Agent Layer
- Database
- External Integrations
- Project Structure
- Getting Started
- Environment Variables
- Running with Docker
- API Reference

🌐 System Overview

The system operates in two modes:

Mode	Description

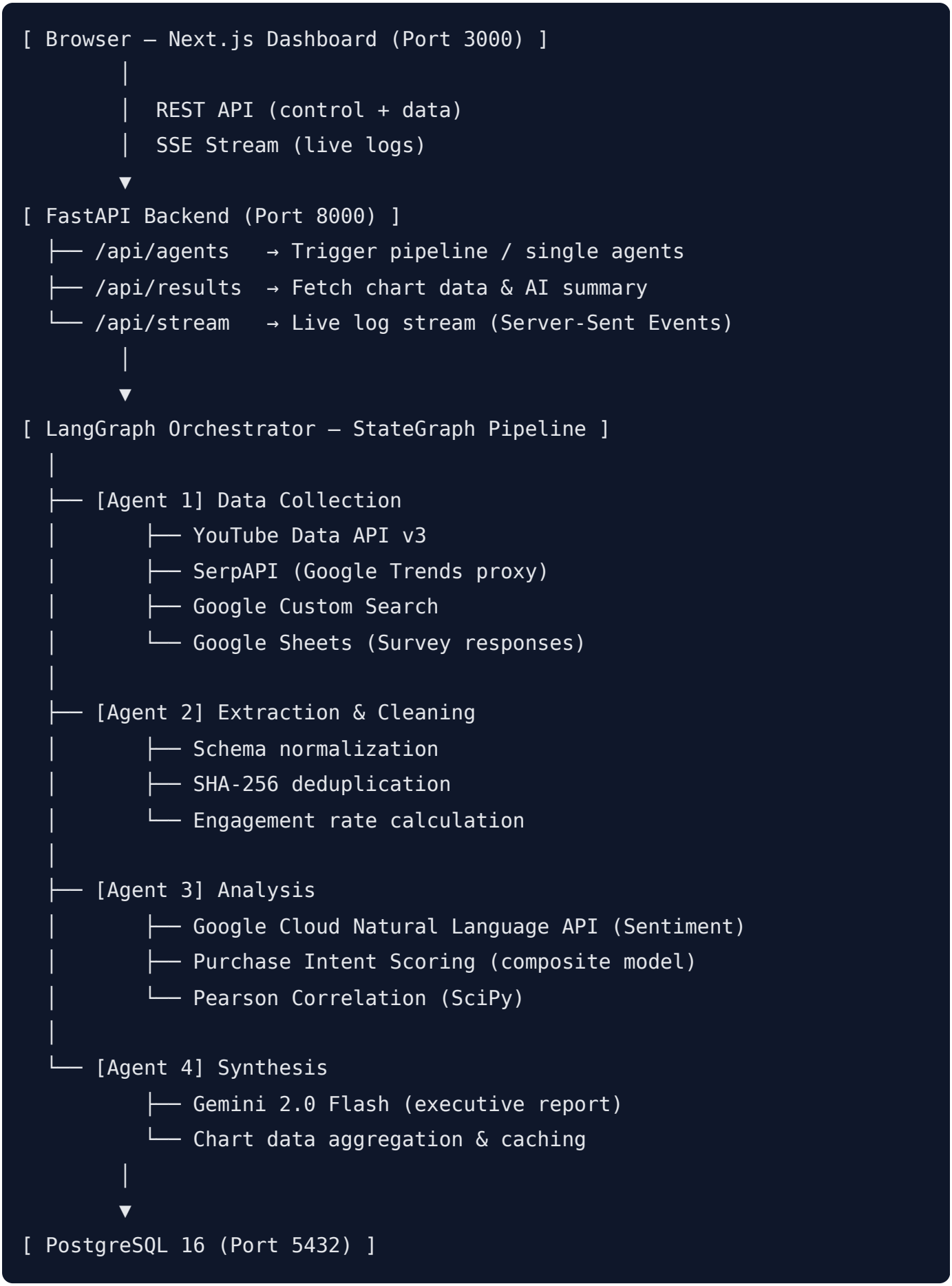
Demo Mode (`MOCK_MODE=true`)	Runs the full pipeline on synthetic data — **no API keys needed**. Perfect for demos and development.
Live Mode (`MOCK_MODE=false`)	Hits real Google Cloud APIs (YouTube, NLP, Trends, Sheets) for genuine research data.

A single button click triggers a four-stage AI pipeline that:

1. Collects digital marketing data from multiple sources
2. Cleans and normalizes the raw data
3. Analyzes sentiment and purchase intent statistically
4. Synthesizes findings into an AI-generated executive report

---

🗺️ Architecture Diagram


[Browser — Next.js Dashboard (Port 3000)]
REST API (control + data)
SSE Stream (live logs)
[FastAPI Backend (Port 8000)]
/api/agents → Trigger pipeline / single agents
/api/results → Fetch chart data & AI summary
/api/stream → Live log stream (Server-Sent Events)
[LangGraph Orchestrator — StateGraph Pipeline]
[Agent 1] Data Collection
YouTube Data API v3
SerpAPI (Google Trends proxy)
Google Custom Search
Google Sheets (Survey responses)
[Agent 2] Extraction & Cleaning
Schema normalization
SHA-256 deduplication
Engagement rate calculation
[Agent 3] Analysis
Google Cloud Natural Language API (Sentiment)
Purchase Intent Scoring (composite model)
Pearson Correlation (SciPy)
[Agent 4] Synthesis
Gemini 2.0 Flash (executive report)
Chart data aggregation & caching
[PostgreSQL 16 (Port 5432)]

🖥️ Frontend

Stack: Next.js 14 (App Router) • React 19 • TypeScript 5 • Tailwind CSS v4 • Framer Motion • Recharts • Lucide React

The frontend is a **single-page iOS-inspired dark dashboard** that gives real-time visibility into every stage of the AI pipeline.

Pages & Layout

File	Purpose
`app/page.tsx`	Root dashboard — orchestrates all state, polling, and SSE connection
`app/layout.tsx`	Root HTML layout with global font and metadata
`app/globals.css`	Global Tailwind base and CSS variables

UI Components

Component	What It Shows
`Header.tsx`	Top bar with product name input, Run/Stop buttons, Demo/Live mode toggle, backend health badge
`MetricCards.tsx`	KPI ribbon: total records, avg sentiment, avg purchase intent, Pearson r value
`PipelineStepper.tsx`	Visual 4-step pipeline status indicator (idle → running → retrying → done/error)
`SummaryCard.tsx`	AI-generated executive summary, key findings, and statistical correlation display
`RecordsExplorer.tsx`	Sortable, filterable table of all analyzed records with platform and sentiment badges
`LogPanel.tsx`	Dual-mode log viewer: live SSE stream + historical DB logs with color-coded severity levels

Chart Components

All charts live in `components/charts/` and are built with **Recharts**:

Chart	Visualization
`SentimentTrendChart.tsx`	Line chart — mean sentiment score over time (by publication date)
`EngagementChart.tsx`	Bar chart — average engagement rate compared across YouTube, Trends, and Survey sources
`PurchaseIntentScatter.tsx`	Scatter plot — engagement rate vs. purchase intent, dots colored by sentiment
`PlatformBreakdown.tsx`	Donut chart — proportion of records per data source platform

Dashboard Tabs

Tab	Content
Analysis & Findings	Summary card + all 4 interactive charts + methodology footnote
Data Records	Full filterable table of every analyzed record
Logs & Pipeline	Live SSE log stream and historical agent run logs

Real-Time Data Flow (Frontend)

```
useEffect → SSEConnection (EventSource)
         ↓
         Live log entries appended to state in real-time

setInterval (2.5s) → api.getStatus()
                   ↓
                   Pipeline agent status badges update

pipelineStatus === "done" → api.getAllResults() + api.getLogs()
                          ↓
                          Charts and summary card refresh automatically
```

⚙️ Backend

Stack: Python 3.11+ • FastAPI 0.111.0 • Uvicorn • Pydantic v2 • sse-starlette • SQLAlchemy 2.0 (asyncio)

The backend is a fully **async FastAPI** server that manages the agent pipeline, exposes REST endpoints, and streams live logs to the dashboard via **Server-Sent Events (SSE)**.

Entry Point

`backend/main.py` — Initializes the FastAPI app, registers CORS middleware (allowing `localhost:3000`), auto-creates database tables on startup via `lifespan`, and mounts all routers.

Routers

Router	Prefix	Responsibility
`routers/agents.py`	`/api/agents`	`POST /run` — starts the full pipeline as a background task; `POST /stop` — cancels it; `GET /status` — returns live pipeline + agent state
`routers/results.py`	`/api/results`	`GET /all` — returns cached synthesis JSON with all chart datasets; `GET /logs` — returns historical agent run logs from DB
`routers/stream.py`	`/api`	`GET /stream` — SSE endpoint that broadcasts real-time log events to all connected browser tabs

Data Models (`models.py`)

Table	Description
`raw_data`	Unmodified API responses from each data source, tagged with `source` and `run_id`
`cleaned_records`	Normalized records with computed `engagement_rate`, `content_hash` (SHA-256), and survey ratings
`analysis_results`	Per-record sentiment scores, purchase intent scores, and platform labels
`agent_run_logs`	Timestamped log entries from each agent per pipeline run

Caching

After every successful run, the Synthesis Agent writes `backend/cache/latest_synthesis.json`. The results router reads this file first — so the dashboard always loads instantly, even with an empty database.

🤖 AI & Agent Layer

The intelligence of the system lives in `backend/agents/`. All four agents are orchestrated as a **LangGraph StateGraph** — a directed graph where each node is an agent and state flows forward through IDs, never raw data blobs.

Orchestrator (`orchestrator.py`)

- Builds and compiles the **LangGraph StateGraph**: `data_collection → extraction_cleaning → analysis → synthesis → END`
- Wraps every agent node in `_node_with_retry`: up to **3 retries** with **exponential backoff** (2s → 4s → 8s)
- On error at any node, uses conditional edges to jump directly to `END` without crashing downstream agents
- Broadcasts all state changes to the SSE log queue non-blockingly

Agent 1 — Data Collection (`data_collection_agent.py`)

What it does: Collects raw marketing signals from four sources and persists them as `RawData` rows.

Source	Data Collected
YouTube Data API v3	Video metadata (title, description, views, likes, comment count) for configured search keywords
SerpAPI / Google Trends	Historical search interest-over-time scores for commercial keywords
Google Custom Search	Web mentions of brand/product names across public pages
Google Sheets	Consumer survey responses: demographics, ad recall, and 1–10 purchase intent ratings

In Mock Mode, all four sources return pre-built synthetic JSON fixtures from `backend/mock_data/`.

Agent 2 — Extraction & Cleaning (`extraction_cleaning_agent.py`)

What it does: Transforms raw API payloads into clean, normalized `CleanedRecord` rows.

Step	Detail
Schema Normalization	Maps source-specific fields (e.g., YouTube `viewCount`) into a unified schema
Date Normalization	Parses and standardizes all timestamp formats to UTC DateTime
SHA-256 Deduplication	Computes a content hash from `(platform + content_id + content_text)` — skips exact duplicates across repeated runs
Engagement Rate	Calculated as `(likes + comments) / views` for YouTube; survey and trends sources use platform-specific proxies

---

Agent 3 — Analysis (`analysis_agent.py`)

What it does: Runs NLP and statistical analysis on every cleaned record, producing `AnalysisResult` rows.

Sentiment Analysis

- **Live Mode:** Calls **Google Cloud Natural Language API v2** (`/documents:analyzeSentiment`) — returns a `score` (−1.0 to +1.0) and `magnitude`.
- **Mock Mode:** Uses a deterministic keyword heuristic with MD5-seeded noise to produce consistent, reproducible results without network calls.
- Labels: `positive` (score ⩾ 0.25), `negative` (score ⩽ −0.25), `neutral` otherwise.

Purchase Intent Score (0–100, weighted composite)

```
40% × normalized sentiment score  (maps −1..1 → 0..100)
35% × normalized engagement rate  (capped at 15% engagement)
25% × survey intent rating        (maps 1..10 → 0..100)
```

Statistical Correlation

- Computes **Pearson's r** and **p-value** (via `scipy.stats.pearsonr`) between `engagement_rate` and `purchase_intent_score` across all records.
- Results are logged in real-time and included in the final synthesis.

Agent 4 — Synthesis (`synthesis_agent.py`)

What it does: Aggregates all analysis results and produces the final report.

Step	Detail
Gemini 2.0 Flash	Calls `gemini-2.0-flash` via `langchain-google-genai` to generate a structured executive summary, key findings, and research insights
Fallback Synthesis	If `GEMINI_API_KEY` is absent or the call fails, a templated academic narrative is generated from the raw statistics
Chart Data Preparation	Aggregates per-platform engagement averages, sentiment time-series, scatter plot points, and platform distribution counts
Cache Write	Writes the complete synthesis payload to `backend/cache/latest_synthesis.json` for instant subsequent reads

---

Shared State (`state.py`)

```
class AgentState(TypedDict):
    run_id: str                    # Unique ID for this pipeline run
    mock_mode: bool                # True = synthetic data
    product_name: Optional[str]    # Target product/brand
    keywords: Optional[list[str]]  # Search keywords
    raw_data_ids: list[str]        # IDs passed from Agent 1 → Agent 2
    cleaned_record_ids: list[str]  # IDs passed from Agent 2 → Agent 3
    analysis_result_ids: list[str] # IDs passed from Agent 3 → Agent 4
    synthesis_output: Optional[dict]
    current_agent: str
    status: str                    # "running" | "done" | "error"
    error_message: Optional[str]
```

> **Key invariant:** Agents pass **database IDs only** — never raw JSON blobs — to keep state small and efficient.

Non-Blocking Log Bus

```
emit_log(agent_name, level, message, run_id)
    ↓ puts entry into asyncio.Queue (LOG_QUEUE)
    ↓ SSE endpoint dequeues and pushes to all browser subscribers
    ↓ Pipeline execution continues — logging never blocks agents
```

🗄️ Database

PostgreSQL 16 with **SQLAlchemy 2.0 asyncio** and **asyncpg** driver.

Tables are auto-created at startup via `Base.metadata.create_all`. Each table uses **UUID primary keys** and is indexed by `run_id` for efficient per-run queries.

Table	Rows Created By
`raw_data`	Data Collection Agent
`cleaned_records`	Extraction & Cleaning Agent
`analysis_results`	Analysis Agent

`agent_run_logs`	All agents via `emit_log()`

🔌 External Integrations

Service	Used By	Key Required
Gemini 2.0 Flash	Synthesis Agent (report generation)	`GEMINI_API_KEY`
Google Cloud Natural Language API	Analysis Agent (sentiment scoring)	`GOOGLE_CLOUD_NLP_API_KEY`
YouTube Data API v3	Data Collection Agent	`YOUTUBE_DATA_API_KEY`
SerpAPI	Data Collection Agent (Google Trends proxy)	`SERPAPI_KEY`
Google Custom Search	Data Collection Agent (brand mentions)	`GOOGLE_CUSTOM_SEARCH_API_KEY` + `GOOGLE_CSE_ID`
Google Sheets API	Data Collection Agent (survey data)	`GOOGLE_SHEETS_SPREADSHEET_ID` + `GOOGLE_SHEETS_CREDENTIALS_PATH`

> All integrations have a Mock Mode fallback — the full pipeline runs without any keys when `MOCK_MODE=true`.

📁 Project Structure

```
mrms_pbl/
├── .env.example              # Environment variable template
├── .env                      # Your local secrets (gitignored)
├── docker-compose.yml        # 3-container setup: postgres + backend +
frontend
├── AGENT.md                  # AI agent session context protocol
│
├── context/                  # Living project documentation (for AI
agents)
│   ├── 01-overview.md        # What the product does and for whom
│   ├── 02-architecture.md    # Tech stack, schema, invariants
│   ├── 03-code-standards.md  # Coding conventions
│   ├── 04-workflow-rules.md  # Agent behavior rules
│   ├── 05-ui-context.md      # Design tokens and UI conventions
│   └── 06-progress.md        # Living progress tracker
│
├── backend/
│   ├── main.py               # FastAPI app entry point
│   ├── database.py           # Async SQLAlchemy engine and session
│   ├── models.py             # ORM: RawData, CleanedRecord,
AnalysisResult, AgentRunLog
│   ├── requirements.txt      # Python dependencies
│   ├── Dockerfile
│   │
│   ├── agents/
│   │   ├── state.py                    # AgentState TypedDict,
PIPELINE_STATE, LOG_QUEUE
│   │   ├── orchestrator.py             # LangGraph StateGraph + retry
wrapper
│   │   ├── data_collection_agent.py    # Agent 1: YouTube, Trends,
Search, Sheets
│   │   ├── extraction_cleaning_agent.py # Agent 2: Normalize,
deduplicate, clean
│   │   ├── analysis_agent.py           # Agent 3: Sentiment, intent
scoring, Pearson r
│   │   └── synthesis_agent.py          # Agent 4: Gemini report + chart
aggregation
│   │
```

```
|   ├── routers/
|   |   ├── agents.py          # Pipeline run/stop/status endpoints
|   |   ├── results.py         # Chart data and summary endpoints
|   |   └── stream.py          # SSE log stream endpoint
|   |
|   ├── mock_data/             # Synthetic JSON fixtures for
MOCK_MODE=true
|   └── cache/
|       └── latest_synthesis.json  # Cached latest run output
|
└── frontend/
    ├── app/
    |   ├── page.tsx           # Root dashboard component
    |   ├── layout.tsx         # HTML shell and metadata
    |   └── globals.css        # Global styles
    |
    ├── components/
    |   ├── Header.tsx          # Top bar: controls, mode toggle, health
badge
    |   ├── MetricCards.tsx     # KPI summary cards ribbon
    |   ├── PipelineStepper.tsx # 4-step visual pipeline tracker
    |   ├── SummaryCard.tsx     # AI executive summary display
    |   ├── RecordsExplorer.tsx # Filterable records data table
    |   ├── LogPanel.tsx        # Live + historical log viewer
    |   └── charts/
    |       ├── SentimentTrendChart.tsx     # Sentiment over time (line)
    |       ├── EngagementChart.tsx         # Engagement by platform (bar)
    |       ├── PurchaseIntentScatter.tsx   # Intent vs. Engagement
(scatter)
    |       └── PlatformBreakdown.tsx       # Platform distribution
(donut)
    |
    ├── lib/
    |   └── api.ts             # Typed API client for all backend
endpoints
    |
    ├── next.config.ts
    └── package.json
```

🚀 Getting Started

Prerequisites

- Node.js 20+
- Python 3.11+
- Docker Desktop *(recommended)*
- PostgreSQL 16 *(if not using Docker)*

Option A — Docker (Recommended)

```
# 1. Clone the repository
git clone <repo-url>
cd mrms_pbl

# 2. Copy and configure environment
cp .env.example .env
# Edit .env — MOCK_MODE=true works out of the box with no API keys

# 3. Start all services
docker compose up --build

# 4. Open the dashboard
# → http://localhost:3000
```

Option B — Manual Setup

```
# --- Backend ---
cd backend
python -m venv venv
venv\Scripts\activate          # Windows
pip install -r requirements.txt

# Start PostgreSQL and set DATABASE_URL in .env, then:
uvicorn main:app --reload --port 8000

# --- Frontend (new terminal) ---
cd frontend
npm install
npm run dev
# → http://localhost:3000
```

🔧 Environment Variables

Copy `.env.example` to `.env` and configure the following:

Variable	Required	Default	Description
`MOCK_MODE`	✅	`true`	`true` = synthetic data, no API keys needed
`DATABASE_URL`	✅	—	PostgreSQL asyncpg connection string
`GEMINI_API_KEY`	Live only	—	Gemini 2.0 Flash for synthesis report

`GOOGLE_CLOUD_NLP_API_KEY`	Live only	—	Sentiment analysis via Cloud NLP
`YOUTUBE_DATA_API_KEY`	Live only	—	YouTube video search & metadata
`SERPAPI_KEY`	Live only	—	Google Trends proxy via SerpAPI
`GOOGLE_CUSTOM_SEARCH_API_KEY`	Live only	—	Brand mention search
`GOOGLE_CSE_ID`	Live only	—	Custom Search Engine ID
`GOOGLE_SHEETS_SPREADSHEET_ID`	Live only	—	Survey data Google Sheet ID
`GOOGLE_SHEETS_CREDENTIALS_PATH`	Live only	`service_account.json`	Path to GCP service account JSON
`NEXT_PUBLIC_API_URL`	✅	`http://localhost:8000`	Backend URL for Next.js

🐳 Running with Docker

The `docker-compose.yml` defines three services:

Service	Image	Port	Notes

`postgres`	`postgres:16-alpine`	5435 → 5432	Persistent volume, healthcheck enabled
`backend`	Custom Python 3.11	8000 → 8000	Waits for postgres health before starting
`frontend`	Custom Node 20	3000 → 3000	Waits for backend to start

```
docker compose up            # Start all services
docker compose up --build    # Rebuild images first
docker compose down -v       # Stop and remove volumes
```

📡 API Reference

Base URL: `http://localhost:8000`
Interactive docs: `http://localhost:8000/docs`

Method	Endpoint	Description
`GET`	`/health`	Health check
`POST`	`/api/agents/run`	Start pipeline (`mock_mode` , `product_name` , `keywords` in body)
`POST`	`/api/agents/stop`	Cancel a running pipeline
`GET`	`/api/agents/status`	Live pipeline + per-agent status
`GET`	`/api/results/all`	Full chart data + AI synthesis summary
`GET`	`/api/results/logs`	Historical agent run logs (query: `limit`)
`GET`	`/api/stream`	SSE stream — real-time log events (`text/event-stream`)
`POST`	`/api/results/clear-cache`	Delete cached synthesis; force fresh results on next load

---

🎓 Academic Context

This project is a **B.Tech Final-Year research workbench** designed to produce empirically sound findings for academic defense. The analytical outputs include:

- **Pearson Correlation Coefficient (r)** between ad engagement and purchase intent
- **Statistical significance (p-value)** at $\alpha = 0.01$ (two-tailed)
- **Sentiment distribution** (positive / neutral / negative) across platforms
- **Composite Purchase Intent Model**: 40% NLP sentiment + 35% engagement rate + 25% survey intent
- **Platform-comparative analysis** across YouTube, Google Trends, web search, and consumer surveys

---

MRMS • Digital Marketing Intelligence & Consumer Behavior Research • Multi-Agent System • Academic Research Workbench